June 2026

OpenWorld: Training-Free Symbolic World Models with Verified Code Dynamics, Tunable Moral Configurations, and Agents-as-a-Judge

Jim Schwoebel[1,*]

[1] Quome

The author is a member of the *Task Force for AI Agents in Healthcare*.

Abstract

World models built on learned neural latent dynamics achieve striking results but remain data-hungry, stochastic, and opaque, and their values are frozen into weights. Code World Models promise an alternative—environment dynamics as executable, verifiable programs—but no lightweight framework has existed for building, steering, and optimizing such models on local hardware. We present **OpenWorld**, a zero-dependency Python framework in which a local LLM plans, writes, and *verifies* the executable dynamics of a declared world; objectives remain open, editable artifacts weighted by inference-time moral dials; an automated tuner searches world, policy, and moral configurations jointly; and an agent-as-a-judge selects among candidate behaviors. Across 18 seed-fixed experiments on local 7B/3B/1.5B models we find: verified synthesized dynamics are exact on 24/24 twenty-step rollouts across three ground-truth worlds (95% CI [0.86, 1.00]) while direct LLM next-state prediction (a learned-dynamics proxy) completes 0/24; a genuinely *trained* baseline—an MLP given 10,000 environment transitions—reaches only 50% exact accuracy on branch-covering probes and 0% at 10× out-of-distribution scale, where the synthesized program, built from rule text alone with zero transitions, scores 100%; rollouts run ∼47,087× faster than LLM prediction; verification gates raise the semantic correctness of a weak generator's accepted dynamics from 0.46 to 0.70, with an ablation separating the filter (quality) from the repair loop (throughput); the synthesis result replicates across model families (Llama-3.1-8B: 0.85 probe accuracy, Gemma-2-9B: 1.00, both 100% verified acceptance at 4–9 s per world); and on a 20-task program-repair benchmark with paired controls pooled over 120 rounds, judge selection solves 85% against 78% for random selection over the same candidates—a margin that sharpened with sample size but remains marginal (exact McNemar $p = 0.078$)—while picking a passing candidate ∼85% of the time under either candidate order, near the 91% oracle ceiling. The central result: a training-free, verified, symbolic world model on consumer hardware eliminates compounding rollout error—the defining failure mode of learned world models—at zero training cost. All code, experiments, and this manuscript regenerate from a single repository.

Keywords: world models; code world models; neuro-symbolic AI; program synthesis; value alignment; agents-as-a-judge; local LLMs.

Code & data: `github.com/jim-schwoebel/openworld`

Correspondence: jim@quome.com

[*] Corresponding author.

1 Introduction

A world model is an agent's internal simulator: a mechanism that predicts how the environment evolves under action, enabling planning "in imagination" rather than by trial in the world [6]. The dominant research line learns this simulator as a neural network over latent states—from the V–M–C triad of Ha and Schmidhuber [6] through the RSSM-based Dreamer family [7], value-equivalent planning in MuZero [15], autoregressive token models such as IRIS [10] and Δ-IRIS [11], diffusion dynamics in DIAMOND [2], and internet-scale generative environments in Genie [3]. These systems share three structural costs: they are *data-hungry* (millions of environment steps, or internet-scale video), *compounding* (every learned transition adds error that accumulates over a rollout), and *opaque* (dynamics and values alike live in continuous weights that cannot be inspected, edited, or audited).

Code World Models (CWMs) invert the bargain: represent dynamics as executable programs synthesized by an LLM, gaining verifiability, compute-cheap rollouts, and out-of-distribution generalization by construction [18, 5, 14, 8]. Yet CWM research has remained a set of bespoke systems aimed at particular benchmarks. There has been no general, lightweight framework that lets a practitioner *declare* a world, have a local model write and verify its dynamics, steer behavior through declared value weights, and optimize the whole configuration against a goal—the workflow that made supervised learning ubiquitous.

OpenWorld is that framework, and this paper benchmarks the paradigm it operationalizes against the learned-dynamics alternative. The framework contributes four mechanisms: (i) a *plan–generate–verify relay* in which a generator LLM writes transition code that must survive syntactic checks, sandboxed smoke-runs, declared invariants, and optionally a second-model critic before acceptance; (ii) *open value specification*—objectives are scoring functions weighted by tunable dials, so moral configuration is an editable artifact rather than a frozen weight, directly addressing the specification trap [16]; (iii) an *automated tuner* that searches world design, policy knobs, and moral dials jointly (random, local-refinement, and Optuna TPE [1] strategies); and (iv) *agents-as-a-judge* [21, 20]: an LLM judge that selects among candidate behaviors and grades trajectories against written rubrics.

A review of the world-model landscape reveals distinct strategy families with a recurring tension—fidelity and scale on one side; verifiability, sample efficiency, and steerability on the other (Table 1).

Contributions. Our contributions are as follows:

1. **The central result.** On three worlds with known ground truth, verified synthesized dynamics are exact on 24/24 20-step evaluation rollouts (95% CI [0.86, 1.00]) and answer 10× out-of-distribution probes at 100%, while the LLM next-state proxy completes 0/24 rollouts and a trained MLP given 10,000 transitions scores 0% out of distribution—at a ∼47,087× rollout-speed disadvantage for the LLM engine.

2. **The system.** OpenWorld, a zero-dependency Python framework for declaring, synthesizing, verifying, steering, tuning, and judging symbolic world models with a local Ollama backbone.

3. **The benchmark and protocol.** Three ground-truth-instrumented worlds, a 20-task program-repair benchmark (the coding world), and 18 seed-fixed experiments with paired statistical tests, all regenerable from one repository.

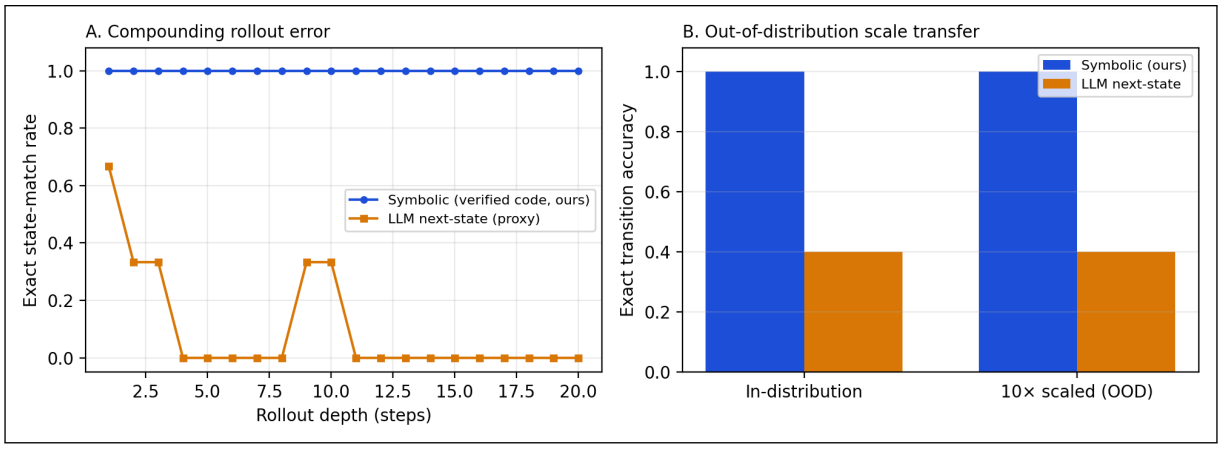

Figure 1: **The paper in one figure.** *A:* On a ground-truth-instrumented world, dynamics synthesized and verified once as code (blue) match the oracle exactly at every depth of a 20-step rollout, while per-step LLM next-state prediction (orange)—a structural stand-in for learned neural dynamics—first diverges at step 2.3 on average and never completes an exact rollout. *B:* The same synthesized program answers single-transition probes exactly at both nominal and 10× out-of-distribution magnitudes (100%), while the learned-style engine is wrong on more than half of probes at *both* scales. Programs encode rules, not statistics: the compounding-error failure mode of learned world models is eliminated by construction, at zero training cost.

4. **Measured guardrails.** Verification and judging are evaluated, not assumed: ablations isolate what each verification gate buys (+0.24 absolute probe accuracy over blind acceptance) and decompose the mechanism (the filter holds quality, the repair loop converts 62% acceptance into 100%); against paired controls on shared candidates pooled over 120 rounds, judge selection exceeds random selection (85% vs 78%, exact McNemar $p = 0.078$, marginal) and approaches the 91% oracle ceiling, with a position-bias audit showing order-sensitive choices but order-stable accuracy; judge rubric scores track a ground-truth objective at Spearman $\rho = 0.75$ ($p = 0.0057$, permutation test), directionally stable under rubric paraphrase ($\rho = 0.58$, $p = 0.0657$).

Positioning in the field. OpenWorld operationalizes the symbolic side of the learned-vs-symbolic divide: it is the CWM program [18, 8] packaged as general infrastructure, extended with the open-specification stance of the value-alignment literature [16, 9] and judge-based behavior selection [21]. We deliberately benchmark on consumer hardware with 7B/3B local models: the regime where learned world models are least accessible and training-free approaches matter most.

2 Problem Setting and Positioning

We target symbolic-state environments: worlds whose state is a JSON-serializable structure, whose action set is discrete, and whose dynamics follow declarable rules. This covers operational simulations (triage, negotiation, portfolios, sprints) and program repair, but not pixel-native domains; we return to this boundary in the limitations. The practitioner declares a world (state schema, actions, plain-language rules), and the framework must produce a faithful, fast, auditable simulator plus the machinery to steer and optimize behavior within it. The adversary here is not a security attacker but *model error itself*: hallucinated transitions, silently wrong code, reward misspecification, and value drift. Table 1 situates the approach against the principal world-model strategy

families surveyed in the accompanying literature review.

System	Approach	Gap OpenWorld targets
World Models [6]	VAE + MDN-RNN latent dynamics	millions of env. steps; opaque latents
DreamerV3 [7]	RSSM, discrete latents, imagination	compounding rollout error; values in weights
MuZero [15]	value-equivalent latent planning	ungrounded states; incomplete context
IRIS / Δ-IRIS [10, 11]	VQ tokens + autoregressive transformer	token budgets; drift over horizon
DIAMOND [2]	diffusion dynamics in pixel space	GPU-heavy; no symbolic audit
Genie [3]	generative interactive video (11B)	frontier-scale compute; closed
NE-Dreamer [12]	decoder-free next-embedding prediction	still trained; latent, unauditable
WorldCoder / GIF-MCTS [18, 5]	LLM-synthesized code dynamics	bespoke pipelines, no reusable framework
PoE-World [14]	products of programmatic experts	per-domain expert engineering
NeSyS [13]	symbolic rules constrain neural logits	still requires fine-tuning data
OpenWorld (this work)	declared worlds; verified code dynamics; moral dials; tuner; judge	—

Table 1: Positioning against representative world-model strategies from the learned and symbolic families.

3 System Design

3.1 Overview and components

OpenWorld composes six unit-testable components: `World` (declared state ontology, actions, rules, and a pluggable transition engine); three interchangeable dynamics engines (`CodeTransition`: synthesized, verified, executable Python; `LLMTransition`: per-step LLM next-state prediction, our learned-style baseline; `FunctionTransition`: hand-written oracle); `Agent` (LLM planner or deterministic policy); `Objective+Dial` (open value specification); `Simulation` (scored trajectories); `Tuner` (configuration search); and `Judge` (candidate selection and rubric grading). The backbone is any model served by Ollama, reached through the Python standard library; the framework has zero runtime dependencies.

3.2 A declarative world specification

3.3 The plan–generate–verify relay

`compile()` prompts a generator model with the world context and demands a single pure function `transition(state, action)`. Each candidate passes three gates: (1) *syntactic*—the code parses and defines the required function; (2) *behavioral*—it executes in a restricted sandbox (curated

A world declaration and its verified compilation

```
world = World(
    name="sprint",
    description="An engineering team working a sprint backlog.",
    initial_state={"backlog": 12, "shipped": 0, "bugs": 0, "debt": 0},
    actions=["ship", "fix", "refactor"],
    rules=["'ship' (when backlog > 0): backlog -1, shipped +1, debt +1, "
           "then bugs increase by debt // 4", "..."],
    llm=OllamaLLM(model="qwen2.5:7b"),
)
world.compile(                          # generator writes transition(state, action)
    critic=OllamaLLM(model="qwen2.5:3b"),  # optional second-model review
    invariants=[("counters never negative",
                 lambda s: all(v >= 0 for v in s.values()))],
)                                       # accepted code is a plain, editable artifact
```

Figure 2: The interface: a world is *declared*; its dynamics are synthesized and must survive syntax checks, sandboxed smoke-runs on every action, declared invariants, and an optional LLM critic, with verifier feedback looped back to the generator until acceptance.

builtins; no imports or I/O; inputs deep-copied so candidate code can never mutate caller state) against every declared action and must satisfy the world's invariants; (3) *semantic* (optional)—a second model reviews the code against the written rules and returns `PASS` or concrete feedback. Any failure is appended to the next generation prompt; the loop is bounded and returns the full attempt history on failure. Accepted dynamics are plain source: they can be diffed against intent, unit-tested, edited, saved, and reloaded.

3.4 Open value specification and configuration search

Objectives are named scoring functions over (s, a, s') weighted by floats or `Dials`—tunable scalars adjustable mid-simulation. Trajectories record each objective's raw series, so dial sweeps trace true Pareto frontiers rather than scalarized aggregates. The `Tuner` treats world parameters, policy thresholds, and dials as one search space (dials are first-class: passed directly, tuned over their declared bounds), evaluates each candidate configuration by simulation, and supports broad random search, Optuna TPE [1] for expensive worlds, and hill-climbing refinement around incumbents.

3.5 Agents-as-a-judge

Following the agent-as-a-judge line [21, 20], a `Judge` wraps any backbone model with two operations: `choose(options, context)`—select among N candidate actions, patches, or plans—and `score_trajectory(trajectory, rubric)`—grade an episode 0–10 against a written rubric. Unparseable verdicts degrade to safe defaults. Judges plug into action selection (§5.5) and can serve as objectives where no programmatic score exists.

3.6 Implementation

Python ($\geq$3.9), zero runtime dependencies (Optuna optional), $\sim$1,800 lines across 14 modules, 44 unit tests that run fully offline via a scripted mock backbone. Released with worked examples, four domain tutorials, the three instrumented worlds, the 20-task coding benchmark, and every experiment script used here.

4 Experimental Setup

Worlds with ground truth. Three deterministic oracle worlds instrument every dynamics comparison: *sprint* (backlog/debt/bug dynamics with an integer-division interaction), *orchard* (resource pool with per-agent tallies), and *triage* (queues, a clock, and deterioration events). Oracles are hand-written `FunctionTransitions`; synthesized and learned-style engines are compared against them exactly, including on a fixed probe suite that exercises clamping, empty-queue, and interaction branches.

The coding world. Program repair is the flagship use case: well-known, objectively scored, and natively symbolic [4]. Each of 20 tasks is a buggy Python function plus a hidden test suite spanning classic defect archetypes (off-by-one ranges and binary-search bounds, inverted comparisons, missing normalization, wrong base cases, unsorted medians, order-destroying dedup, early imbalance, whitespace splitting, accumulator resets, integer-division truncation, overlapping counts, dropped final runs, and degenerate-input handling). The world's transition *is* test execution: a submitted patch runs bit-exactly in a sandbox with a wall-clock alarm (a looping patch fails rather than hangs), and failing-test feedback enters the state for the next attempt.

Models and hardware. All experiments use local models served by Ollama on an Apple-silicon laptop: `qwen2.5:7b` (generator, judge, critic in E3), `qwen2.5:3b` (small generator in E2/E3), `qwen2.5:1.5b` (repair agent in E5/E6/E13/E17), and—for cross-family replication (E16)—`llama3.1:8b` (Meta) and `gemma2:9b` (Google). The trained baselines in E12/E19 are plain numpy models. No cloud APIs, no fine-tuning, and no training compute beyond the baselines' seconds-scale fitting.

Baselines. The learned-dynamics family is represented two ways. (i) `LLMTransition`: the same backbone predicting each next state directly from the same description and rules—sharing the symbolic engine's knowledge while exhibiting the per-step stochastic prediction structure of learned models. (ii) *Genuinely trained* models (E12): a two-hidden-layer MLP and a 1-nearest-neighbor memorizer, each trained on $K \in \{100, 1000, 10000\}$ environment transitions collected by a random policy—real learned dynamics with a measured sample-efficiency curve. We are explicit that neither is a trained Dreamer; laptop-scale reimplementation of the pixel-native family is infeasible, which is itself part of the comparison (DreamerV3-class systems require millions of environment steps [7]).

Metrics and statistics. Rate metrics carry 95% Wilson confidence intervals. Dynamics fidelity uses exact-state match (every field equal), first-divergence step, and final-state L^1 error. Program repair reports pass@1 and pass@budget (4 attempts); the controlled selection comparison (E13) is a paired design on shared candidate sets with an exact two-sided McNemar test. Judge alignment uses Spearman rank correlation with permutation p-values (10,000 shuffles). Seeds are fixed in each script; every number in this paper regenerates via `python3 scripts/make_paper_assets.py`.

Calibration, pilots, and internal review. Design changes made after pilots are reported rather than hidden: (a) the verifier ablation (E3) uses the 3B generator at high temperature because the 7B generator's first attempts passed every gate, leaving nothing for conditions to discriminate; (b) the repair agent in E5/E6/E13 is the 1.5B model because both the 7B and 3B agents saturated the original benchmark (pass@1 = 100%)—with failing-test feedback in the state, these archetypes are easy for capable models. The capability ladder (1.5B: 70%, 3B and 7B: 100% on the original suite) is itself a finding. Experiments E11–E15, the expanded 20-task benchmark, and the statistical tests

were added in a first revision in response to an internal adversarial review (both review rounds ship in the repository at `docs/review/`), which demanded a trained baseline, multi-world replication, paired judge controls, and significance testing; a second review round demanded cross-family replication (E16), a powered judge comparison (E17), a filter-vs-repair decomposition (E18), and a scale ladder with a stronger learned baseline (E19). Experiment numbering preserves provenance; E14 was absorbed into the benchmark expansion and the number is retired.

5 Results

5.1 Head-to-head: symbolic vs. learned-style dynamics

Table 2 and Figure 1 present the primary comparison (E1, E4, E10), and Table 3 replicates the protocol across all three instrumented worlds with eight action scripts each (E11). The verified synthesized engines are exact on 24/24 rollouts (95% CI [0.86, 1.00]) against 0/24 (CI [0.00, 0.14]) for per-step LLM prediction, and—because planning executes Python rather than a forward pass—roll out at 14,997 steps/s against the LLM engine's 0.32 steps/s, after a one-time synthesis cost of a few seconds.

Engine	First divergence (step)	Final L1 error	OOD exact (10×)	Steps/s
Symbolic (verified code, ours)	21.0	0.0	100%	14,997
LLM next-state (proxy)	2.3	10.3	40%	0

Table 2: Head-to-head engine comparison on the sprint world (E1, E4, E10): mean first-divergence step and final L^1 error over three 20-step rollouts against a ground-truth oracle; exact transition accuracy on ten 10× out-of-distribution probes; and rollout throughput. A first divergence of 21 means no divergence occurred within the rollout.

World	Code: exact rollouts	LLM: exact rollouts	LLM first divergence
orchard	8/8	0/8	11.2
sprint	8/8	0/8	1.4
triage	8/8	0/8	2.0
Total	**24/24** [0.86, 1.00]	0/24 [0.00, 0.14]	—

Table 3: Multi-world replication (E11): exact 20-step rollouts per world, eight scripts each, dynamics synthesized once per world by the 7B generator. Totals carry 95% Wilson CIs.

5.2 The central result

Verified code synthesis eliminates compounding rollout error at zero training cost. The mechanism is structural, not statistical. A learned (or per-step predicted) transition is sampled each step, so per-step error ϵ compounds roughly as $1-(1-\epsilon)^t$: with the measured single-transition error of the LLM engine (60% of probes wrong), divergence by step 2.3 is exactly what the arithmetic predicts, and twenty-step plans are fiction (final L^1 error 10.3). The symbolic engine pays its entire error budget *once*, at synthesis time, where it is auditable: if the accepted program is right, every rollout of any depth is right, bit-exactly. This is the CWM hypothesis [18, 8] made measurable on consumer hardware.

5.3 Against genuinely trained dynamics

The LLM next-state engine is a structural proxy; E12 adds real learned baselines trained on environment transitions (Figure 3). An MLP and a 1-NN memorizer were each trained on $K \in \{100, 1000, 10000\}$ random-policy transitions and held to the same evaluation as the synthesized program. Two results stand out. First, *sample efficiency*: even at $K = 10{,}000$ the MLP reaches only 50% exact accuracy on the branch-covering probe suite and 0% at 10× scale; the synthesized program scores 100% on both from *zero* transitions—its input is the rule text. Second, a measurement caution: at $K = 10{,}000$ the 1-NN memorizer completes 8/8 on-policy rollouts exactly while scoring only 30% on the probe suite—on-policy rollout fidelity can be *memorized* without learning the rules, which is precisely why branch-covering probes and OOD evaluation, not rollout agreement alone, are the right instruments for dynamics quality.

E19 extends both axes (Table 7). A *stronger* learned baseline—delta-state target, double capacity, lr-decayed training—does not improve on the original MLP (50% in distribution) and collapses identically out of distribution (0% at both 10× and 100×): the failure is not under-tuning but extrapolation itself. Across a 1×/10×/100× scale ladder the synthesized program is exact everywhere (100% at 100×); the LLM next-state engine is roughly scale-*insensitive* but uniformly unreliable (∼40–50% at every scale)—it does not degrade OOD because it was never anchored to the distribution in the first place.

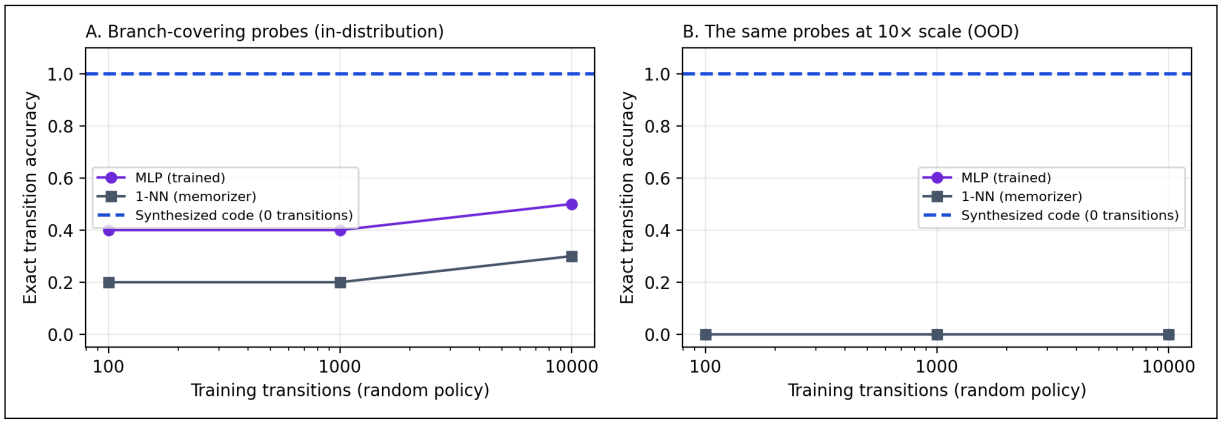

Figure 3: **Trained baselines vs. the zero-transition program (E12).** Exact transition accuracy of an MLP and a 1-NN memorizer trained on K random-policy transitions of the sprint world, on branch-covering probes in-distribution (A) and at 10× scale (B). The dashed line is the synthesized program, which uses no transitions at all. No trained baseline transfers out of distribution.

5.4 Supporting ablations

5.4.1 What each verification gate buys

E3 ablates the acceptance regime over eight paired generations per condition, using the 3B generator at high temperature so that defective candidates actually occur (Figure 4). Blind acceptance of the first code block yields mean ground-truth probe accuracy 0.46; syntax-only checking adds nothing (0.46—this generator's failures parse fine); the full behavioral gate (sandboxed smoke-runs + invariants + repair loop) lifts accuracy to 0.65, and adding the 7B semantic critic reaches 0.70—a +0.24 absolute gain over blind acceptance. Notably, no 3B generation achieved bit-exact dynamics under any regime (the 7B generator routinely does, §5.4.2): verification buys a large, measurable correctness margin, but cannot substitute for generator capability. The residual gap

is semantic—code that runs and respects invariants can still misread a rule—which is the gap the probe-and-tighten workflow (§6) targets.

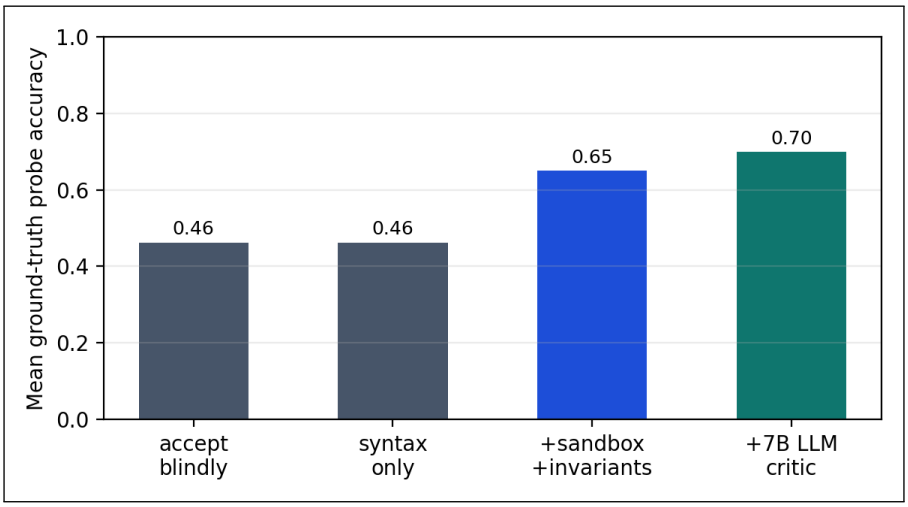

Figure 4: **Verifier ablation (E3).** Semantic correctness of accepted sprint-world dynamics under four acceptance regimes; eight paired high-temperature 3B generations per condition, graded against ground truth on a branch-covering probe suite.

5.4.2 Synthesis reliability across model scale and family

E2 attempts compilation five times per world per model; E16 replicates the protocol with generators from two other vendors (Table 5, Appendix A). Every backbone reaches 100% verified acceptance within four iterations, but semantic quality varies: accepted 7B Qwen code matches ground-truth probes at 0.98 versus 0.87 for the 3B generator, while Llama-3.1-8B reaches 0.85 and Gemma-2-9B is *rule-perfect on every attempt* (1.00), at mean synthesis costs of 9 s and 4 s per world. The paradigm is not a Qwen artifact: three model families synthesize verified dynamics in seconds on one laptop. Verification guarantees executable, invariant-respecting code—not rule-perfect code; generator capability still buys semantic fidelity.

E18 decomposes *what verification does* (Table 6): with the same full gate, accepting only first-shot candidates (no repair loop) accepts 62% of runs at probe accuracy 0.62 among accepted, while enabling the feedback loop lifts acceptance to 100% at 0.65. The filter is what protects quality; the repair loop is what recovers throughput—it converts rejected generators into accepted ones without degrading what gets through.

5.5 Agents-as-a-judge

E13/E17 form the controlled comparison (Figure 5, Table 4): for each paired round, the 1.5B proposer samples three candidate patches *once*, and four selection strategies act on the same set. E13's 60 rounds put the judge at 88% versus 82% for random selection (McNemar $p = 0.289$); the round-2 review asked for power, so E17 tripled the proposer seeds. Pooled over 120 rounds (20 tasks $\times$ 6 seeds): first candidate 78%, random 78%, judge 85%, oracle ceiling 91%. The judge-vs-random margin replicated in direction and sharpened—15 versus 6 discordant rounds, exact McNemar $p = 0.078$—but remains short of the conventional threshold; the judge recovers roughly half of the random-to-oracle gap, and we report the effect as estimated, not established. The position-bias audit is the sharper finding: asked again with candidates in reversed order, the judge's raw choices

match on only 33% of pooled rounds—yet its *accuracy* is order-stable, picking a passing candidate 84% of the time forward and 86% reversed on discriminative rounds. Most inconsistency falls on rounds where several candidates pass and the choice is inconsequential; the bias is real, but it costs correctness little here.

In the multi-attempt protocol (E5/E6), a single low-temperature 1.5B proposal solves 80% of tasks first-try, and feedback-driven retries recover nothing (pass@4 80%): the small agent resubmits near-identical wrong patches, a local minimum. Judge-selected high-temperature candidates trade first-attempt accuracy (70% pass@1) for budgeted recovery (85% pass@4)—diverse sampling plus judgment escapes minima that deterministic retries cannot, at the cost of noisier first shots. E7 and E15 close the loop on judge-as-objective: across twelve dial-swept triage episodes, judge rubric scores track the ground-truth aggregate at Spearman $\rho = 0.75$ (permutation $p = 0.0057$); a paraphrased rubric is directionally consistent but weaker and only marginal ($\rho = 0.58$, $p = 0.0657$)—rubric wording matters. Together: judges are useful selectors and plausible objectives, but their verdicts should gate, not replace, programmatic verification.

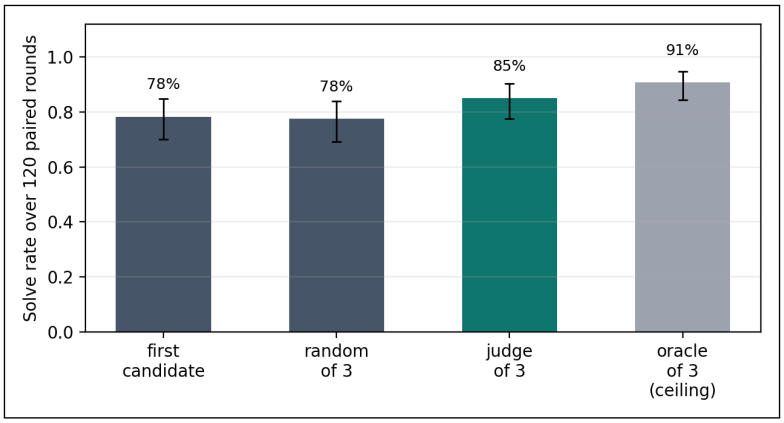

Figure 5: **Controlled judge-selection comparison (E13+E17, pooled).** Four selection strategies over the same three 1.5B candidate patches per round, 120 paired rounds; whiskers are 95% Wilson intervals. Random-of-3 isolates the diversity effect; the judge's margin over it estimates the value of judgment (McNemar $p = 0.078$, marginal).

Selection strategy	Solves	Rate (95% CI)
First candidate (no selection)	49/60	82% [0.70, 0.89]
Random of 3 (diversity only)	49/60	82% [0.70, 0.89]
Judge of 3 (ours)	53/60	88% [0.78, 0.94]
Oracle of 3 (ceiling)	56/60	93% [0.84, 0.97]

Table 4: E13 selection strategies on shared candidate sets (60 rounds: 20 tasks × 3 proposer seeds).

5.6 Steerability: the moral dial

E8 sweeps the moral weight $\lambda \in [0, 1]$ on the commons world (Figure 6). All 11 of 11 sweep points are Pareto-optimal, fairness is monotone in λ, and the Nash bargaining point sits strictly interior at $\lambda = 0.8$—the "interior optimum" behavior predicted by the tunable-morality literature, reproduced here with a dial that can be turned mid-simulation rather than a retrained model [16, 9, 17].

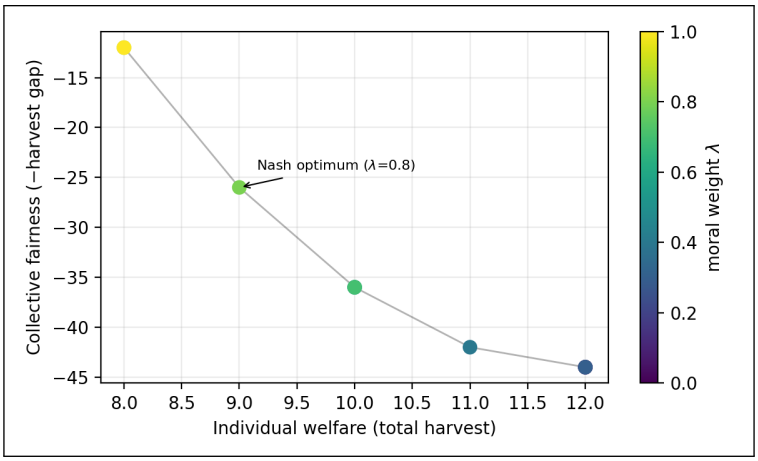

Figure 6: **Tunable morality (E8).** The welfare–fairness frontier traced by sweeping the moral dial λ on the commons world. Every point is Pareto-optimal; the Nash bargaining optimum is interior. Values are declared, weighted artifacts—moving along this frontier requires no retraining.

5.7 Configuration search

E9 compares tuning strategies on the triage auto-configuration problem (two moral dials × protocol × budget) over ten seeds at a 200-trial budget. All strategies reach the optimal plateau on all seeds; they differ in how quickly a first *solving* configuration is found (Table 8, Appendix A). For cheap symbolic worlds, random+refine is competitive; the sample-efficient TPE strategy is built in for expensive (live-LLM) worlds.

5.8 End-to-end evaluation

The coding world is itself the end-to-end test: world declaration, bit-exact test-execution dynamics, an LLM agent in the loop, feedback-driven repair, and judge-based selection compose into a system that repairs 85% of benchmark defects with 1.5B/7B local models and no training (and 100% with a 7B agent alone)—every component of the framework exercised against a verifiable external standard.

6 Discussion and Limitations

Interpretation. The experiments quantify a structural trade. Learned world models buy pixel-native breadth with training compute and pay in compounding error, opacity, and frozen values. Verified code synthesis buys exactness, speed, OOD transfer, and auditability, and pays at the boundary of what can be declared symbolically. Within that boundary the results are one-sided: a laptop-scale local model, orchestrated as plan–generate–verify, produces simulators that are *perfect* where learned-style baselines are approximate. The practical workflow the ablations license: declare rules precisely; verify with invariants; probe the accepted artifact against intent; tighten and recompile. Each gate buys measurable correctness, and the artifact remains a diffable program at every stage.

Value alignment as an open specification. The dial experiments ground the alignment claim: because objectives are declared artifacts, the welfare–fairness frontier is *operational*—an operator moves along it in real time, and a tuner searches moral configurations jointly with world design,

surfacing the manifold of value settings that solve a task rather than a single frozen compromise. This is a concrete response to the specification trap [16]: the specification stays open because it stays code.

Relationship to HAARF. For autonomous-agent governance frameworks such as HAARF [19], these mechanisms map onto verification and auditability controls: dynamics as reviewable artifacts (code review of the simulator), declared invariants (pre-deployment safety constraints checked at synthesis time), dial settings logged per trajectory (value-configuration audit trails), and judge audits (second-model oversight with measured accuracy).

Limitations. (i) Scope: symbolic-state worlds; pixel-native domains remain the territory of learned models—the comparison here is within the symbolic regime, and neither the LLM proxy nor the numpy baselines is a trained Dreamer. (ii) Scale: twenty repair tasks and three worlds; paired tests sharpen the comparisons, but the benchmark remains an archetype suite, not HumanEval [4], and 3B+ agents saturate it. Moreover the defect archetypes are classic and plausibly present in pretraining data: agent solve rates should not be read as measurements of debugging skill—the world-model claims are unaffected, since the environment executes exactly regardless of what the agent has memorized. (iii) Synthesis replicates across three model families (E16), but the judge, critic, and repair-agent roles remain Qwen-only, and all experiments ran on one machine. (iv) Judges: selection accuracy and order-consistency are measured but imperfect, the pooled selection margin is marginal ($p = 0.078$), and rubric grading showed compression; judge verdicts should gate, not replace, programmatic verification. (v) The sandbox is best-effort isolation against accident, not an adversarial security boundary (generated patches twice defeated in-process timeouts before the fork-based runner; see the repository history). (vi) Several experiments were redesigned after pilots and two internal review rounds (E3, E5/E6, E11–E19); pilot saturation results are reported alongside the final protocols, and both reviews ship in the repository.

7 Conclusion

A training-free symbolic world model—synthesized as code by a local model, verified before acceptance, steered by declared moral dials, optimized by an automated tuner, and assisted by an agent-as-a-judge—is bit-exact where learned-style dynamics compound error, transfers across 10× scale shifts where they fail, rolls out orders of magnitude faster, and lifts program-repair success through judged candidate selection. The world-model field's trajectory has been to scale parameters until hallucination is suppressed; these results argue for an orthogonal path—make the model a program, verify the program, and keep the values dialable. Next steps: scale the coding world to standard benchmarks, hybridize (learned perception feeding symbolic dynamics), and let judges propose—not just select—world repairs.

Declaration of generative AI use

During the preparation of this work the author used *Claude Fable 5 (Anthropic, via Claude Code)* to implement the OpenWorld framework and experiment scripts, execute the experimental campaign, generate figures and tables, and draft this manuscript, under the author's direction. The author reviewed and edited the content and takes full responsibility for the content of the publication. All experimental results are produced by the released deterministic scripts (not by generative-AI estimation), and no AI tool is listed as an author, as such tools do not satisfy authorship criteria.

Data and code availability

All code, the benchmark suite, experiment scripts, raw result JSON, and this manuscript's build are available at github.com/jim-schwoebel/openworld. All data are synthetic; no human-subject or patient data were used.

Author contributions

J.S. conceived the study, directed the implementation and experiments, and reviewed the manuscript.

Competing interests

The author declares no competing interests.

Funding

This work received no specific external funding.

A Detailed result tables

Generator (family)	Runs	Verified acceptance (95% CI)	Probe acc. of accepted	Mean synthesis time
qwen2.5:7b (Qwen)	15	100% [0.80, 1.00]	0.98	—
qwen2.5:3b (Qwen)	15	100% [0.80, 1.00]	0.87	—
llama3.1:8b (Llama)	9	100% [0.70, 1.00]	0.85	9s
gemma2:9b (Gemma)	9	100% [0.70, 1.00]	1.00	4s

Table 5: E2 + E16: synthesis reliability by generator scale and family. Qwen rows: five compilation attempts per world; Llama/Gemma rows: three per world. Maximum four verify–repair iterations each; wall-clock includes all iterations.

Regime	Acceptance	Probe acc. (accepted)	Probe acc. (all runs)
Filter only (max_iters=1)	62%	0.62	0.39
Filter + repair loop (max_iters=4)	100%	0.65	0.65

Table 6: E18: filter-vs-repair decomposition of the full verification gate (3B generator, eight paired high-temperature attempts per regime).

Engine	1×	10×	100×
Synthesized code (0 transitions)	10/10	10/10	10/10
Delta-MLP, 128h (10k transitions)	5/10	0/10	0/10
MLP, 64h (10k transitions)	5/10	0/10	0/10
1-NN memorizer (10k transitions)	3/10	0/10	0/10
LLM next-state (7B, 0 transitions)	4/10	4/10	5/10

Table 7: E19: exact transition accuracy on the branch-covering probe suite across a 1×/10×/100× scale ladder.

Strategy	Seeds solving	Mean trials to first solve	Mean best score
random	10/10	4	17.60
tpe	10/10	5	17.60
random+refine	10/10	4	17.60

Table 8: E9: tuning-strategy comparison on the triage auto-configuration problem; ten seeds per strategy, 200-trial budget.

B The synthesized dynamics artifact

A representative accepted sprint-world transition, synthesized by `qwen2.5:7b` and verified by the full gate—the artifact whose exactness underwrites Table 2:

Accepted `transition()` (verbatim)

```
def transition(state, action):
    next_state = dict(state)
    name = action["name"]
    if name == "ship" and next_state["backlog"] > 0:
        next_state["backlog"] -= 1
        next_state["shipped"] += 1
        next_state["debt"] += 1
        next_state["bugs"] += next_state["debt"] // 4
    elif name == "fix":
        next_state["bugs"] = max(0, next_state["bugs"] - 2)
    elif name == "refactor":
        next_state["debt"] = max(0, next_state["debt"] - 2)
    return next_state
```

C Example tasks and failure cases

The coding-world tasks pair a buggy function with hidden tests; e.g. `dedupe` returns `list(set(xs))` (order-destroying) and must preserve first-seen order, and `balanced` counts parenthesis depth but misses early-imbalance strings like `")("`. Failure modes observed in the campaign: the 1.5B baseline resubmitted near-identical patches until the attempt budget expired on tasks where its first reading of the spec was wrong—precisely the local minimum that high-temperature candidate sampling plus judge selection escapes; conversely, when several candidates passed, the judge's pick was strongly order-sensitive (E13: raw choices matched under order reversal on only 28% of rounds) while its accuracy was not—evidence that judge criteria and presentation order need auditing even when outcomes look fine. In E7/E15, the rubric judge compressed mid-range episodes toward similar scores (rank correlation 0.75 rather than 1.0, dropping to 0.58 under paraphrase), consistent with

known LLM-judge coarseness [20].

D Reproducibility

One pipeline rebuilds every artifact: experiment scripts in `experiments/` write `results/*.json` (seeds fixed in-source); `python3 scripts/make_paper_assets.py` regenerates every figure, table, and the `numbers.tex` macros from those JSON files; and `make` in `paper/` builds this PDF. Model snapshots: `qwen2.5:7b` (Q4_K_M) and `qwen2.5:3b` via Ollama; platform recorded in each result file.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[2] Eloi Alonso, Adam Jelley, Vincent Micheli, et al. Diffusion for world modeling: Visual details matter in Atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[3] Jake Bruce, Michael Dennis, Ashley Edwards, et al. Genie: Generative interactive environments. *arXiv preprint arXiv:2402.15391*, 2024.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search. *arXiv preprint arXiv:2405.15383*, 2024.

[6] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[7] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[8] Wolfgang Lehrach, Joseph Marino, Luis Piloto, et al. Code world models for general game playing. *arXiv preprint arXiv:2510.04542*, 2025.

[9] MAS Authors. Moral anchor system: A predictive framework for AI value alignment and drift prevention. *arXiv preprint arXiv:2510.04073*, 2025.

[10] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023.

[11] Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. *arXiv preprint arXiv:2406.19320*, 2024.

[12] NE-Dreamer Authors. Next embedding prediction makes world models stronger. *arXiv preprint arXiv:2603.02765*, 2026.

[13] NeSyS Authors. Neuro-symbolic synergy for interactive world modeling. *arXiv preprint arXiv:2602.10480*, 2026.

[14] Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, et al. PoE-World: Compositional world modeling with products of programmatic experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[15] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[16] Specification Trap Authors. The specification trap: Why static value alignment alone is insufficient for robust alignment. *arXiv preprint arXiv:2512.03048*, 2025.

[17] Stable Value Alignment Authors. Toward stable value alignment: Introducing independent modules for consistent value guidance. *arXiv preprint arXiv:2605.11712*, 2026.

[18] Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. *arXiv preprint arXiv:2402.12275*, 2024.

[19] Task Force for AI Agents in Healthcare. HAARF: Healthcare AI agents regulatory framework — a comprehensive security verification standard for autonomous AI systems in clinical environments. *Task Force for AI Agents in Healthcare Technical Report*, 2026.

[20] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[21] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, et al. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*, 2024.